Why Are You Tired?

The original ELIZA reused your own words: "I am tired" came back as "Why are you tired?" Today ELIZA learns to take your sentence apart – and to keep a log.

Marcus Birkenkrahe

COSC 1210 · CS I · Augustana University

Fall 2026 · Downey, *Think Python* 3e, ch. 8

Review: mini challenge (10 min)

→ **Switch to Colab · birkenkrahe.github.io/go/replyreview · 10 min**

```
def respond(sentence):
    keyword = find_keyword(sentence)
    if keyword == "sorry":
        return "Please don't apologize."
    ...
    elif keyword == "always":
        return "Can you think of a specific example?"
    else:
        return "Please go on."
```

```
>>> is_family("mother")
True
>>> is_family("dream")
False
>>> respond("My brother never calls.")
'Tell me more about your family.'
```

1. Write a Boolean function `is_family(keyword)`: `True` if `keyword` is one of the four family words in `keywords.txt` – "mother", "father", "sister", "brother" – else `False`. One `return`, no `if`.

   Then add one `elif` branch to `respond` with the condition `is_family(keyword)`: "Tell me more about your family." Leave `keywords.txt` as it is.

2. A shorter version:
   `return keyword in "mother father sister brother"`
   What does it return for `"other"` and for `None` – and why?

Review: mini challenge – solution

1. One return, one new branch

```
def is_family(keyword):
    return (keyword == "mother"
            or keyword == "father"
            or keyword == "sister"
            or keyword == "brother")

def respond(sentence):
    ...
    elif is_family(keyword):
        return "Tell me more about your family."
    ...
```

```
True
False
Tell me more about your family.
```

2. With in

```
def is_family_short(keyword):
    return (keyword in
        "mother father sister brother")

print(is_family_short("other"))
print(is_family_short(None))
```

```
True
TypeError: 'in <string>' requires
string as left operand, not NoneType
```

Review: mini challenge – discussion

- The parentheses let the condition run over two lines.
- The comparison is already `True` or `False`: return it, no `if`.
- With `in`, "other" and "her" count as family: `in` finds substrings.
- `is_family_short(None)` crashes: `in` on a string needs a string on the left, and `None` is not a string (its type is `NoneType`). In `respond` that would happen for every sentence without a keyword – `find_keyword` returns `None`.
- `is_family(None)` is safe: == compares `None` with a string and gives `False`, so `respond` says "Please go on."
- What is a doctest, and how do you run one?
  → An example in the docstring: a >>> line with a call, then the line it should print. `run_doctests` runs every >>> line and compares; it prints only the examples that do not match.

Review: catch the bug with a doctest (5 min)

→ Switch to Colab · same notebook, part 5 · 5 min

```
from doctest import run_docstring_examples

def run_doctests(func):
    run_docstring_examples(func, globals(),
                           name=func.__name__)
```

1. Three doctests in `is_family`: `"mother"` → `True`, `"dream"` → `False`, `"other"` → `False`
2. The same three in `is_family_short`
3. Run `run_doctests` on both. Which one prints something?

`is_family`: no output – all three pass.

`is_family_short`:

```
File "__main__", line 23, in is_family_short
Failed example:
    is_family_short("other")
Expected:
    False
Got:
    True
```

→ The test catches the bug only because it has the "other" example. With just "mother" and "dream", both versions pass.

A string is a sequence

→ **Switch to editor · code-along 1**

```
sentence = "hello, world!"
print(sentence[0])
print(sentence[2])
print(len(sentence))
print(sentence[-1])
```

```
h
l
13
!
```

- Each character has a position, its **index**, starting at 0
- The index goes in square brackets
- The last index is `len(sentence) - 1`, here 12 – or simply −1: negative indexes count from the end

```
+---+---+---+---+---+---+---+---+---+---+---+---+---+
| h | e | l | l | o | , |   | w | o | r | l | d | ! |
+---+---+---+---+---+---+---+---+---+---+---+---+---+
  0   1   2   3   4   5   6   7   8   9  10  11  12
-13 -12 -11 -10  -9  -8  -7  -6  -5  -4  -3  -2  -1
```

Slices: “I am X” → “Why are you X?”

```
sentence = "i am tired"
print(sentence[:4])
print(sentence[5:])
```

```
i am
tired
```

`[n:m]`: from index `n` up to, not including, `m`. Leave out `n` or `m` to start at the beginning or go to the end.

```
def i_am_reply(sentence):
    sentence = sentence.lower()
    if sentence.startswith("i am "):
        return f"Why are you {sentence[5:]}?"
    return None

print(i_am_reply("I am tired of homework"))
print(i_am_reply("You are tired"))
```

```
Why are you tired of homework?
None
```

`"i am "` has 5 characters: the rest starts at index 5. `startswith` checks the beginning first.

Two surprises

→ Switch to editor · TRY (2 min) · predict, then run

```
print(i_am_reply("I am tired."))
print(i_am_reply("I am what I am"))
```

```
Why are you tired.?
Why are you what i am?
```

→ The period is part of the slice. Code-along 3 removes it.

→ The slice copies "i am" as it is: ELIZA should say "what you are". Swapping the speaker's words is called reflection – code-along 4.

Strings cannot be changed

→ **Switch to editor · code-along 2**

```
reply = "why are you tired?"
reply[0] = "W"
```

```
TypeError: 'str' object does not support
item assignment
```

Strings are **immutable**. Build a new one:

```
reply = "W" + reply[1:]
print(reply)
```

```
Why are you tired?
```

Comparing strings:

```
print("apple" < "banana")
print("Zebra" < "apple")
```

```
True
True
```

```
print(f"{ord('a')} < {ord('b')}")
```

```
97 < 98
```

String methods

→ **Switch to editor · code-along 3**

```
print("Why me?".endswith("?"))
print("No way!!!".count("!"))
print("I'm tired".replace("I'm", "I am"))
print("tired.".strip("."))
```

```
True
3
I am tired
tired
```

- A **method** is called with a dot. String methods never change the string – they return a new one
- `endswith`: the partner of `startswith`
- `count`: how often a substring occurs
- `replace`: every occurrence of the first string by the second
- `strip(".!?")`: removes those characters at both ends; without an argument, whitespace

Normalize the input

```
def normalize(sentence):
    sentence = sentence.lower()
    sentence = sentence.replace("i'm", "i am")
    return sentence.strip(".!?")

def i_am_reply(sentence):
    sentence = normalize(sentence)
    if sentence.startswith("i am "):
        return f"Why are you {sentence[5:]}?"
    return None

print(i_am_reply("I am tired."))
print(i_am_reply("I'm tired!"))
```

```
Why are you tired?
Why are you tired?
```

Normalizing: bring what people type into one standard form. Then ELIZA needs to check only one form.

Chaining: each method returns a new string.

```
return (sentence.lower()
        .replace("i'm", "i am")
        .strip(".!?"))
```

Reflection: “my” becomes “your”

→ **Switch to editor · code-along 4**

```
print(i_am_reply("I am angry at my sister."))
```

```
Why are you angry at my sister?
```

ELIZA should say “your sister”. Turning the speaker’s words around is **reflection**. Try `replace`:

```
sentence = "i like you"
sentence = sentence.replace("i", "you")
print(sentence)
sentence = sentence.replace("you", "i")
print(sentence)
```

```
you lyouke you
i like i
```

Two problems with `replace`

1. `replace` works on letters, not words: the "i" inside "like" is replaced too
2. The second `replace` cannot tell the old "you" from the new one: both become "i"
3. TRY (2 min): would it help to swap the two `replace` calls?

→ No: then every "i" becomes "you", and "like" still breaks.

Reflection has to go **word by word**, looking up each word once: lists (chapter 9) and a dictionary (chapter 10).

```
def reflect(sentence):
    swaps = {"i": "you", "am": "are", "my": "your"}
    words = []
    for word in sentence.lower().split():
        words.append(swaps.get(word, word))
    return " ".join(words)

print(reflect("i am angry at my sister"))
print(reflect("I don't know who I am"))
```

```
you are angry at your sister
you don't know who you are
```

ELIZA keeps a log

→ **Switch to editor · code-along 5**

The keyword setup cell: `open("keywords.txt", "w")` opens the file for **writing – file mode** `"w"`.

```
log = open("log.txt", "w")
log.write("You: I am tired.\n")
log.write("ELIZA: Why are you tired?\n")
log.close()

for line in open("log.txt"):
    print(line.strip())
```

```
You: I am tired.
ELIZA: Why are you tired?
```

- `write` returns the number of characters written – that is the number the setup cell showed in the notebook
- `write` adds no newline: write the `\n` yourself
- `close` finishes the file. Text not yet closed may not be in the file

"w" starts over, "a" appends

→ **Switch to editor · TRY (2 min): open log.txt with "w" again, write one line, print the file**

```
log = open("log.txt", "w")
log.write("You: I'm tired!\n")
log.close()

for line in open("log.txt"):
    print(line.strip())
```

```
You: I'm tired!
```

Mode "w" empties the file first. Mode "a" **appends**.

```
def log_exchange(sentence, reply):
    log = open("log.txt", "a")
    log.write(f"You: {sentence}\n")
    log.write(f"ELIZA: {reply}\n")
    log.close()

open("log.txt", "w").close()
log_exchange("I'm tired!", i_am_reply("I'm tired!"))
log_exchange("I am angry.", i_am_reply("I am angry."))
```

```
You: I'm tired!
ELIZA: Why are you tired?
You: I am angry.
ELIZA: Why are you angry?
```

Practice: do you often feel that way?

→ **Switch to editor · 10 min, then 5 min discussion**

Write `feel_reply(sentence)`: for a sentence that starts with "I feel", return "Do you often feel X?", where X is the rest of the sentence. For any other sentence, return `None`. Use `normalize` and a slice.

```
>>> print(feel_reply("I feel tired."))
Do you often feel tired?
>>> print(feel_reply("I feel happy today!"))
Do you often feel happy today?
>>> print(feel_reply("You feel cold."))
None
```

Checkout

1. `"i am tired."[5:]` is `"tired."`. Why is the period still there, and how does ELIZA get rid of it?

→ A slice keeps everything up to the end. `strip(".!?")` in `normalize` removes it.

2. What happens to the old contents of a file that you open with mode `"w"`?

→ They are deleted. Mode `"a"` adds to the end instead.

Summary

- A string is a sequence: `s[0]` is the first character, `s[-1]` the last. A slice `s[n:m]` runs from index `n` up to, not including, `m`
- Strings are **immutable**: build a new string with `+`, a slice or an f-string
- `==`, `<`, `>` compare strings; uppercase letters come before lowercase ones
- String methods return new strings: `lower`, `startswith`, `endswith`, `count`, `replace`, `strip`. **Normalizing** the input makes ELIZA's checks simpler
- `replace` works on letters, not words, and cannot swap two words. **Reflection** has to go word by word
- `open(name, "w")` writes a new file, `"a"` appends to it; `write` needs its own `\n`; `close` finishes the file

Glossary

#	Term	Meaning
1	sequence	Ordered collection, each item at an index
2	index	Position in a sequence, starting at 0
3	slice	Part of a sequence, `s[n:m]`
4	immutable	Cannot be changed after it is made
5	normalize	Bring input into one standard form
6	reflection	Swap the speaker's words: "my" to "your"
7	file mode	How a file is opened: `"r"`, `"w"` or `"a"`
8	append	Add to the end, e.g. mode `"a"`

Code glossary

#	Code	Meaning
1	`s[i]`	Character at index i
2	`s[-1]`	Last character
3	`len(s)`	Number of characters in s
4	`s[n:m]`	Characters from index n up to, not including, m
5	`s.startswith(x)`	True if s begins with x
6	`s.endswith(x)`	True if s ends with x
7	`s.count(x)`	How often x occurs in s
8	`s.replace(a, b)`	New string, every a replaced by b
9	`s.strip(".!?")`	New string without these characters at both ends
10	`s.lower()`	New string in lowercase
11	`s.lower().strip()`	Chaining: each method works on the new string
12	`ord(c)`	Code number of character c
13	`open(name, "w")`	Open a file for writing: starts over
14	`open(name, "a")`	Open a file for appending: adds to the end
15	`f.write(text)`	Write text to file f, no newline added
16	`f.close()`	Close file f

Next: patterns

`i_am_reply` and `feel_reply` are the same function with a different prefix.

- Next: **regular expressions** describe a whole family of sentences in one pattern – "I am X", "I need X", "... my X ..."
- and fix the "smothered" problem on the way